CSCI 5523 Introduction to Data Mining

Spring 2026

Assignment 0 (2 points)

Deadline: January. 28th 11:59 PM CDT

1.Overview of the Assignment

In Assignment 0, you will complete a single task. The goal of HW0 is to verify that Spark is correctly installed and that you can perform basic data transformations using PySpark.

Important: This is a bonus assignment that serves as a technical "check" to assess your readiness for the rest of the course. If this assignment takes more than three hours to complete, it is a strong indication that you may have difficulty finishing the remaining assignments in this class on time.

2. Requirements

2.1 Programming Requirements

a. You must use Python to implement all tasks. You can only use standard Python libraries (e.g., math), but **external libraries like numpy or pandas are not allowed**. If you have questions about specific packages, please ask on Piazza.
b. **You are required to use only Spark RDD;** there is no point in using Spark DataFrame or DataSet.

2.2 Submission Platform

We will use Gradescope to run and grade your submission automatically. You need to first test your scripts on your local machine before submitting your solutions.

2.3 Programming Environment

Python 3.9.12 and Spark 3.2.1

You will need to have Java installed to run PySpark. We tested the environment with JDK 11. Please note that PySpark requires Java 8 (but will not work with versions prior to update 8u371), 11 or 17.

We recommend using either Miniconda or Anaconda to create a virtual environment to run your code in. To install miniconda or anaconda, you may reference the following links:
https://docs.anaconda.com/miniconda/install/ and https://docs.anaconda.com/anaconda/install/

Once conda is installed, you can reference the following link to install pyspark:
https://spark.apache.org/docs/latest/api/python/getting_started/install.html#using-conda

2.4 Write Your Own Code

Do not use large language models (e.g., ChatGPT, Copilot) or share code with other students!

For this assignment to be an effective learning experience, **you must write your own code**. We emphasize this point because you will be able to find Python implementations of some of the required functions on the web. Please do not look for or at any such code.

TAs will combine all the code we can find from the web (e.g., GitHub), other students' code from this and other (previous) sections, and code generated from AI tools for plagiarism detection. **We will report all detected plagiarism to the university.**

3.Tasks

You need to submit the following files (all lowercase):

Python scripts: **`task1.py`**

3.1. Dataset

The input dataset (**gpa.json**) is provided in JSON format, where each line is a valid JSON object representing a single student record. Each record contains the following fields: student_id, name, gpa. You can access and download the datasets from the "data" folder on Google Drive. **We generated these datasets using random sampling. These given datasets are only for your testing. During the grading period, we will use different sampled subsets of the datasets.**

3.2. Task description

In this task, you will verify that your Spark environment is correctly set up by running a simple PySpark program. You will load a JSON-lines student dataset, use basic RDD transformations and actions to extract GPA values and compute their average, and then save the result to a JSON output file.

3.3. Execution commands

You must implement **`task1.py`** from scratch and submit your own completed Python file. The program must follow the required command-line interface described below. You must use the **argparse** library to ensure that **task1.py** can be executed using command-line arguments in the specified format. Please do not change the names of any required command-line parameters. You may write helper functions if needed, but you may import only standard Python libraries.

Your **task1.py** should be executable using the following command line:

```python
python task1.py --input_file <input_file> --output_file <output_file>
```

Params:

`input_file` – the input JSON file
`output_file` – the output file contains your answer

3.4. Output format:

The output must be written in JSON format with the following structure:

`{ "avg_gpa": <average_gpa_value> }`

The value of **avg_gpa** must be a numeric value (float). The output file must contain only this JSON object. The output file is saved in the same directory as **task.py** and must be named **ans_hw0.json.**

4. Grading Criteria

(% penalty = % penalty of possible points you get)

1. You can use your free 5-day extension separately or together. During grading, TAs will count the number of late days **based on the submission time.** For example, if the due date is 2026/02/24 23:59:59 CDT and your submission is 2026/02/25 05:23:54 CDT, the number of late days would be 1. **TAs will record grace days by default (NO separate Emails)**. However, if you want to save it for next time and treat your assignment as a late submission (with some penalties), **please leave a comment in your submission.**
2. No points will be given if your submission cannot be executed on Gradescope.
3. There is no re-grading. Once the grade is posted on Canvas, we will only re-grade your assignments if there is a grading error. No exceptions.
4. Homework assignments are due at 11:59 pm CT on the due date and should be submitted on Canvas. Late submissions within 24 hours of the due date will receive a 30% penalty. Late submissions after 24 hours of the due date will receive a 70% penalty. Every student has FIVE free late days for homework assignments. You can use these free late days for any reason, separately or together, to avoid the late penalty. There will be no other extensions for any reason. **You cannot use the free late days after the last day of the class.**

5. Submission Instructions

1. You will submit your programming assignment via Gradescope. You can access it from Canvas.
2. On Gradescope, you will find a submission link. This link is for submitting **a single** Python file (**`.py`**).
3. For Python Codes: you need to submit a single file (lowercase): **`task1.py`**
4. You are allowed to submit your code to Gradescope as long as it is before the due date. If you submit your codes after the due dates, they will be recorded as late submissions (See Section 4 for information about grading criteria.